Eventos, delegación e interacción en JavaScript

> **ⓘ Enfoque de la sesión**
>
> En esta sesión vas a trabajar la parte de JavaScript que convierte una interfaz renderizada en una interfaz **realmente interactiva**. El objetivo no es solo aprender a detectar clics o cambios, sino comprender cómo se articula el flujo completo de interacción: ocurre un evento, tu código lo interpreta, decide qué hacer y actualiza la interfaz en consecuencia.

1. Introducción

En la sesión anterior trabajaste con asincronía, promesas, `fetch()` y una primera organización del código para trabajar con datos que llegan más tarde. Eso te permitió entender que una interfaz puede depender de información externa y que el flujo de una aplicación no siempre es lineal ni inmediato.

Sin embargo, una interfaz no está completa solo porque pueda renderizar datos o recibirlos desde una fuente externa. Una aplicación web real necesita además **reaccionar a las acciones de la persona usuaria**.

Cuando alguien:

- hace clic en un botón;
- escribe en un campo;
- cambia una opción;
- pulsa un control dentro de una tarjeta;
- o interactúa con una lista dinámica;

se produce un **evento**. JavaScript puede escuchar esos eventos y ejecutar lógica en respuesta.

Esa capacidad es uno de los pilares del desarrollo frontend. Gracias a ella puedes:

- abrir y cerrar paneles;
- filtrar elementos;
- alternar estados visuales;
- eliminar o modificar partes de una lista;
- responder a controles renderizados dinámicamente;

- preparar formularios;
- y coordinar la interacción con los datos y con el DOM.

Pero escuchar eventos no basta. También necesitas organizar bien la interacción. En una interfaz medianamente dinámica suelen intervenir al menos tres piezas:

- el **evento** que ocurre;
- la **lógica** que decide qué hacer;
- la **actualización visual** que modifica el DOM.

Comprender esa secuencia con claridad es esencial para escribir código mantenible y será una preparación directa para React, donde volverás a pensar una y otra vez en flujos del tipo:

- sucede algo;
- se interpreta la acción;
- cambia el estado o los datos;
- la interfaz se actualiza.

---

2. Qué es un evento

2.1. Definición general

Un evento es una señal de que ha ocurrido algo en el documento o en el navegador. Puede estar provocado por la interacción de una persona usuaria o por otros cambios del entorno.

Ejemplos habituales:

- un clic;
- la escritura en un input;
- el envío de un formulario;
- el cambio de una opción;
- el paso del ratón por un elemento;
- la carga de un recurso;
- una tecla pulsada.

2.2. Qué hace JavaScript con los eventos

JavaScript puede suscribirse a esos eventos y ejecutar una función cuando ocurren.

```
const boton = document.querySelector("#boton-saludo");
```

```
boton.addEventListener("click", () => {
  console.log("Se ha hecho clic");
});
```

2.3. Idea clave de la sesión

Un evento no es una “acción mágica” del navegador. Es simplemente una notificación a la que tu código puede responder.

> **Idea clave**
>
> La interfaz no solo se construye.
> También debe **escuchar**, **interpretar** y **responder**.

Referencias específicas del subtema

- MDN Web Docs — Introduction to events
- MDN Web Docs — Event

---

3. Escuchar eventos con `addEventListener()`

3.1. Sintaxis básica

La forma recomendada de escuchar eventos es mediante `addEventListener()`.

```
<button id="boton-saludo">Saludar</button>
```

```
const boton = document.getElementById("boton-saludo");

boton.addEventListener("click", () => {
  console.log("Hola");
});
```

3.2. Partes de la llamada

Elemento	Función
elemento DOM	nodo que va a escuchar el evento

Elemento	Función
tipo de evento	por ejemplo, `"click"`
callback	función que se ejecuta cuando ocurre el evento

3.3. Por qué `addEventListener()` es la opción preferible

Frente a otras formas más antiguas de asignar eventos, `addEventListener()`:

- separa mejor estructura y comportamiento;
- permite añadir varios listeners al mismo elemento;
- es la base del trabajo habitual actual en JavaScript.

3.4. Qué debes evitar al empezar

Un error frecuente consiste en ejecutar una función al registrarla, en lugar de pasarla como callback.

```
const boton = document.querySelector("#boton");

// Incorrecto si querías que se ejecute al hacer clic
boton.addEventListener("click", mostrarMensaje());
```

En ese caso la función se ejecuta inmediatamente.

Referencias específicas del subtema

- MDN Web Docs — `EventTarget.addEventListener()`

---

4. Eventos frecuentes en interfaz

4.1. `click`

Es el evento más habitual en botones, enlaces o elementos interactivos.

```
const boton = document.querySelector("#abrir-panel");

boton.addEventListener("click", () => {
  console.log("Panel abierto");
});
```

4.2. `input`

Se dispara cuando cambia el valor de un campo mientras se escribe o se modifica su contenido.

```
<input id="busqueda" type="text">
```

```
const busqueda = document.querySelector("#busqueda");

busqueda.addEventListener("input", () => {
  console.log("Valor actualizado");
});
```

4.3. `change`

Suele utilizarse cuando se quiere reaccionar al cambio confirmado de un valor, por ejemplo en `select`, `checkbox` o ciertos controles de formulario.

```
<select id="filtro">
  <option value="todos">Todos</option>
  <option value="destacados">Destacados</option>
</select>
```

```
const filtro = document.querySelector("#filtro");

filtro.addEventListener("change", () => {
  console.log("Se ha cambiado el filtro");
});
```

4.4. `submit`

Se usa con formularios y será especialmente importante en la sesión siguiente.

```
<form id="formulario-busqueda"></form>
```

```
const formulario = document.querySelector("#formulario-busqueda");

formulario.addEventListener("submit", () => {
  console.log("Formulario enviado");
});
```

4.5. `mouseover` y `mouseout`

Pueden usarse para detectar cuándo el puntero entra o sale de un elemento.

```
const tarjeta = document.querySelector(".tarjeta");

tarjeta.addEventListener("mouseover", () => {
  console.log("Ratón sobre la tarjeta");
});
```

Referencias específicas del subtema

- MDN Web Docs — Element: `click` event
- MDN Web Docs — Element: `input` event
- MDN Web Docs — HTMLElement: `change` event
- MDN Web Docs — HTMLFormElement: `submit` event

---

5. El objeto evento

5.1. Qué es

Cuando se dispara un evento, JavaScript proporciona un objeto con información sobre lo ocurrido. Ese objeto se suele recibir como parámetro de la función callback.

```
const boton = document.querySelector("#boton-saludo");

boton.addEventListener("click", (evento) => {
  console.log(evento);
});
```

5.2. Utilidad práctica

El objeto evento te permite saber, entre otras cosas:

- qué tipo de evento ha ocurrido;
- qué elemento ha lanzado el evento;
- información adicional según el tipo de interacción.

5.3. `target`

`target` hace referencia al elemento sobre el que se produjo el evento.

```
const lista = document.querySelector("#lista");

lista.addEventListener("click", (evento) => {
  console.log(evento.target);
});
```

Esto será especialmente útil más adelante para trabajar con delegación de eventos.

5.4. `currentTarget`

`currentTarget` representa el elemento al que está asociado el listener actual.

```
const lista = document.querySelector("#lista");

lista.addEventListener("click", (evento) => {
  console.log("target:", evento.target);
  console.log("currentTarget:", evento.currentTarget);
});
```

5.5. Diferencia conceptual inicial

- `target` apunta al origen del evento;
- `currentTarget` apunta al elemento que está escuchando en ese momento.

> **Criterio importante**
>
> Esta diferencia parece pequeña, pero es muy importante cuando trabajas con elementos anidados o con delegación de eventos.

Referencias específicas del subtema

- MDN Web Docs — Event
- MDN Web Docs — `Event.target`
- MDN Web Docs — `Event.currentTarget`

---

6. Flujo de interacción: evento, lógica y actualización visual

6.1. La secuencia completa

En una interfaz, reaccionar a un evento no debería consistir solo en “hacer algo”. Conviene entender el flujo:

1. ocurre un evento;
2. se identifica qué ha pasado;
3. se decide qué lógica aplicar;
4. se actualiza la interfaz.

6.2. Ejemplo simple

```html
<button id="boton-activar">Activar</button>
<p id="estado">Inactivo</p>
```

```javascript
const boton = document.querySelector("#boton-activar");
const estado = document.querySelector("#estado");

boton.addEventListener("click", () => {
  estado.textContent = "Activo";
});
```

6.3. Por qué importa este enfoque

Pensar así te ayudará a no mezclar todo de manera desordenada y a organizar mejor interacciones más complejas.

Referencias específicas del subtema

- MDN Web Docs — Introduction to events
- MDN Web Docs — `Node.textContent`

---

7. Propagación de eventos

7.1. Qué significa que un evento se propaga

Cuando se produce un evento en un elemento, este no queda aislado únicamente en ese nodo. Puede recorrer parte del árbol DOM.

Esto explica por qué una interacción sobre un botón dentro de una tarjeta puede ser detectada también por elementos contenedores.

7.2. Captura y burbujeo

En términos generales, el flujo suele explicarse mediante dos fases:

- **captura**, desde los niveles superiores hacia el elemento objetivo;
- **burbujeo**, desde el elemento objetivo hacia arriba.

En la práctica cotidiana inicial, la idea más útil es entender bien el **burbujeo**.

7.3. Ejemplo de burbujeo

```html
<div id="contenedor">
  <button id="boton-interno">Haz clic</button>
</div>
```

```javascript
const contenedor = document.querySelector("#contenedor");
const boton = document.querySelector("#boton-interno");

contenedor.addEventListener("click", () => {
  console.log("Clic en el contenedor");
});

boton.addEventListener("click", () => {
  console.log("Clic en el botón");
});
```

Al hacer clic en el botón, verás que se ejecutan ambos listeners.

7.4. Por qué ocurre

Porque el evento se origina en el botón y luego burbujea hacia sus ancestros.

> **Idea clave**
>
> La propagación no es un error del navegador.
> Es un comportamiento normal del sistema de eventos y resulta muy útil cuando lo entiendes bien.

Referencias específicas del subtema

- MDN Web Docs — Event bubbling
- MDN Web Docs — Event capture

---

8. `stopPropagation()`

8.1. Qué hace

`stopPropagation()` impide que el evento siga propagándose a otros elementos superiores.

```
const contenedor = document.querySelector("#contenedor");
const boton = document.querySelector("#boton-interno");

contenedor.addEventListener("click", () => {
  console.log("Clic en el contenedor");
});

boton.addEventListener("click", (evento) => {
  evento.stopPropagation();
  console.log("Clic solo en el botón");
});
```

8.2. Cuándo puede ser útil

Puede ser útil cuando:

- necesitas evitar una reacción del contenedor;
- quieres aislar un comportamiento concreto;
- hay varias capas de interacción y debes controlar el flujo.

8.3. Precaución de diseño

No conviene usar `stopPropagation()` como solución automática a cualquier problema. A veces el problema real está en una mala organización de listeners o en una estrategia de interacción poco clara.

> ⚠ **Criterio de uso**
>
> `stopPropagation()` es una herramienta útil, pero no debe sustituir a una buena comprensión del flujo de eventos.

Referencias específicas del subtema

- MDN Web Docs — `Event.stopPropagation()`

---

9. Delegación de eventos

9.1. Qué problema resuelve

Imagina una lista dinámica con muchos botones de eliminar. Una posibilidad sería añadir un listener individual a cada botón. Sin embargo, si esos botones se crean o cambian dinámicamente, esa estrategia se vuelve incómoda y frágil.

La delegación de eventos resuelve esto aprovechando la propagación: colocas un único listener en un contenedor y detectas desde allí qué elemento concreto originó la interacción.

9.2. Ejemplo básico

```html
<ul id="lista-tareas">
  <li><button class="eliminar">Eliminar</button></li>
  <li><button class="eliminar">Eliminar</button></li>
</ul>
```

```javascript
const lista = document.querySelector("#lista-tareas");

lista.addEventListener("click", (evento) => {
  if (evento.target.classList.contains("eliminar")) {
    console.log("Se ha pulsado un botón de eliminar");
  }
});
```

9.3. Por qué es tan importante

La delegación de eventos:

- reduce la cantidad de listeners;
- funciona mejor con elementos generados dinámicamente;
- simplifica la gestión de interacción en listas;
- encaja muy bien con interfaces donde la estructura cambia.

9.4. Uso de `closest()` con delegación

En muchos casos no basta con detectar el botón, sino que necesitas localizar el elemento contenedor relacionado.

```html
<ul id="lista-tareas">
  <li class="tarea">
    <span>Estudiar eventos</span>
```

```
    <button class="eliminar">Eliminar</button>
  </li>
</ul>
```

```
const lista = document.querySelector("#lista-tareas");

lista.addEventListener("click", (evento) => {
  if (evento.target.classList.contains("eliminar")) {
    const tarea = evento.target.closest(".tarea");
    console.log(tarea);
  }
});
```

9.5. Delegación como idea de diseño

La delegación de eventos no es solo una técnica concreta. También es una forma más madura de pensar la interacción: en lugar de reaccionar desde muchos puntos dispersos, centralizas la escucha y decides la lógica según el origen real del evento.

> **Idea clave**
>
> La delegación de eventos es una de las herramientas más útiles para trabajar con listas dinámicas y estructuras que cambian en tiempo de ejecución.

Referencias específicas del subtema

- MDN Web Docs — Event bubbling
- MDN Web Docs — `Event.target`
- MDN Web Docs — `Element.closest()`

---

10. Interacción sobre elementos dinámicos

10.1. Qué significa

Un elemento dinámico es aquel que no estaba necesariamente en el HTML inicial, sino que se ha creado después mediante JavaScript.

10.2. Problema habitual

Si renderizas una lista nueva y no organizas bien los eventos, puedes encontrarte con que los nuevos botones o controles no responden como esperabas.

10.3. Solución conceptual

La delegación de eventos ayuda mucho en este escenario porque no depende de añadir listeners a cada nodo recién creado.

10.4. Ejemplo orientativo

```
const lista = document.querySelector("#lista-tareas");

function agregarTarea(texto) {
  const li = document.createElement("li");
  li.classList.add("tarea");
  li.innerHTML = `
    <span>${texto}</span>
    <button class="eliminar">Eliminar</button>
  `;
  lista.append(li);
}
```

Si el listener está en `lista`, los nuevos botones quedan cubiertos por la estrategia delegada.

10.5. Matiz importante sobre `innerHTML`

En este ejemplo `innerHTML` se usa solo como atajo para ilustrar la idea general de elemento dinámico. En desarrollos más cuidados, conviene seguir priorizando `createElement()`, `textContent()` y `append()` cuando quieras un mayor control sobre la estructura y el contenido.

Referencias específicas del subtema

- MDN Web Docs — `Element.append()`
- MDN Web Docs — Event bubbling

---

11. Cambios visuales con `classList.toggle()`

11.1. Qué hace

`classList.toggle()` añade una clase si no está presente y la elimina si ya existe.

```html
<button id="boton-menu">Mostrar menú</button>
<nav id="menu" class="oculto"></nav>
```

```javascript
const botonMenu = document.querySelector("#boton-menu");
const menu = document.querySelector("#menu");

botonMenu.addEventListener("click", () => {
  menu.classList.toggle("oculto");
});
```

11.2. Cuándo es útil

Es especialmente útil para:

- mostrar y ocultar paneles;
- activar estados visuales;
- alternar clases de una interfaz interactiva;
- representar un estado simple de visible/oculto o activo/inactivo.

11.3. Relación con el diseño de interacción

En muchos casos, la lógica no consiste en reconstruir completamente el DOM, sino en **cambiar el estado visual** de elementos ya existentes.

Referencias específicas del subtema

- MDN Web Docs — `Element.classList`

---

12. Separar datos, eventos y renderizado

12.1. El problema de mezclarlo todo

Cuando una misma función:

- escucha un evento;
- modifica directamente varios nodos;
- cambia datos internos;
- decide visualmente demasiadas cosas;
- elimina y crea elementos;
- contiene validaciones y mensajes;

el código tiende a volverse difícil de mantener.

12.2. Organización más clara

Conviene distinguir conceptualmente:

- los **datos** que representan el estado;
- los **eventos** que desencadenan cambios;
- la **función de renderizado** que actualiza la interfaz.

12.3. Ejemplo sencillo de estructura

```
const tareas = [
  { id: 1, titulo: "Estudiar DOM", completada: false },
  { id: 2, titulo: "Practicar eventos", completada: false }
];

const lista = document.querySelector("#lista-tareas");

function renderizarTareas() {
  lista.innerHTML = "";

  tareas.forEach((tarea) => {
    const li = document.createElement("li");
    li.classList.add("tarea");

    if (tarea.completada) {
      li.classList.add("completada");
    }

    li.innerHTML = `
      <span>${tarea.titulo}</span>
      <button class="toggle" data-id="${tarea.id}">Cambiar estado</button>
    `;

    lista.append(li);
  });
}

lista.addEventListener("click", (evento) => {
  if (evento.target.classList.contains("toggle")) {
    const id = Number(evento.target.dataset.id);
    const tarea = tareas.find((item) => item.id === id);

    if (tarea) {
```

```
      tarea.completada = !tarea.completada;
      renderizarTareas();
    }
  }
});

renderizarTareas();
```

12.4. Qué interés tiene este enfoque

Aunque aquí todavía no estás trabajando con React, ya aparece una idea muy cercana:

- hay datos;
- ocurre una interacción;
- cambia algo en esos datos;
- la interfaz se vuelve a renderizar.

> **Idea clave**
>
> Cuanto mejor distingas entre datos, eventos y renderizado, más clara será tu forma de construir interfaces.

Referencias específicas del subtema

- MDN Web Docs — Introduction to events
- MDN Web Docs — `Document.querySelector()`
- MDN Web Docs — `Element.classList`

---

13. Eventos personalizados con `CustomEvent`

13.1. Qué es un evento personalizado

Además de los eventos nativos del navegador, JavaScript permite crear eventos propios para comunicar acciones internas de una aplicación.

13.2. Ejemplo simple

```
const tarjeta = document.querySelector("#tarjeta");

tarjeta.addEventListener("productoSeleccionado", (evento) => {
```

```
  console.log("Producto recibido:", evento.detail);
});

const evento = new CustomEvent("productoSeleccionado", {
  detail: { id: 1, nombre: "Teclado" }
});

tarjeta.dispatchEvent(evento);
```

13.3. Por qué conviene conocerlo

En esta fase no necesitas trabajar en profundidad con eventos personalizados, pero sí conviene saber que existen como mecanismo para comunicar acciones más allá de los eventos nativos del navegador.

Referencias específicas del subtema

- MDN Web Docs — `CustomEvent`
- MDN Web Docs — `EventTarget.dispatchEvent()`

---

14. Análisis técnico avanzado

14.1. Un listener mal colocado puede complicar toda la interacción

No basta con “escuchar el evento correcto”. También importa **dónde** lo escuchas. Una mala ubicación de listeners puede producir duplicidades, comportamiento confuso o código innecesariamente rígido.

14.2. La propagación es una ventaja cuando sabes aprovecharla

Al principio puede parecer molesta, porque hace que un clic afecte también a elementos superiores. Sin embargo, esa misma propagación permite implementar delegación de eventos y simplificar mucho la arquitectura de interacción.

14.3. `target` y `currentTarget` expresan perspectivas distintas del mismo evento

Comprender bien esta diferencia ayuda a interpretar mejor lo que está ocurriendo en interfaces anidadas y es muy importante cuando el listener está en un contenedor pero el origen real es un elemento hijo.

14.4. No toda interacción exige rehacer toda la interfaz

A veces basta con alternar una clase o modificar un texto. Otras veces conviene rerenderizar un bloque completo. Elegir bien entre ambos enfoques depende del problema y de la organización del código.

14.5. La delegación prepara muy bien el pensamiento de componentes

Aunque React abstrae el manejo del DOM directo, la idea de centralizar comportamientos y pensar la interacción como flujo controlado es muy valiosa y prepara el salto a componentes, handlers y actualizaciones de estado.

Referencias específicas del subtema

- MDN Web Docs — Event bubbling
- MDN Web Docs — `Event.target`
- MDN Web Docs — `Event.currentTarget`
- MDN Web Docs — `Event.stopPropagation()`
- MDN Web Docs — `CustomEvent`

---

15. Ejemplo integrador de sesión

A continuación tienes un ejemplo que reúne varios conceptos trabajados.

```html
<section>
  <button id="boton-filtro">Mostrar/Ocultar filtro</button>
  <div id="panel-filtro" class="oculto">
    <p>Opciones de filtrado</p>
  </div>

  <ul id="lista-tareas"></ul>
</section>
```

```javascript
const botonFiltro = document.querySelector("#boton-filtro");
const panelFiltro = document.querySelector("#panel-filtro");
const lista = document.querySelector("#lista-tareas");

const tareas = [
  { id: 1, titulo: "Repasar arrays", completada: false },
  { id: 2, titulo: "Practicar DOM", completada: true },
  { id: 3, titulo: "Estudiar eventos", completada: false }
];
```

```
function renderizarTareas() {
  lista.innerHTML = "";

  tareas.forEach((tarea) => {
    const li = document.createElement("li");
    li.classList.add("tarea");
    li.dataset.id = tarea.id;

    if (tarea.completada) {
      li.classList.add("completada");
    }

    li.innerHTML = `
      <span>${tarea.titulo}</span>
      <button class="toggle">Cambiar estado</button>
      <button class="eliminar">Eliminar</button>
    `;

    lista.append(li);
  });
}

botonFiltro.addEventListener("click", () => {
  panelFiltro.classList.toggle("oculto");
});

lista.addEventListener("click", (evento) => {
  const tarea = evento.target.closest(".tarea");

  if (!tarea) {
    return;
  }

  const id = Number(tarea.dataset.id);
  const indice = tareas.findIndex((item) => item.id === id);

  if (indice === -1) {
    return;
  }

  if (evento.target.classList.contains("toggle")) {
    tareas[indice].completada = !tareas[indice].completada;
    renderizarTareas();
  }
```

```
  if (evento.target.classList.contains("eliminar")) {
    tareas.splice(indice, 1);
    renderizarTareas();
  }
});

renderizarTareas();
```

Qué se está aplicando aquí

Elemento	Uso en el ejemplo
`addEventListener()`	escucha de clics en botón y lista
`classList.toggle()`	mostrar u ocultar el panel
Renderizado dinámico	construcción de tareas a partir de datos
Delegación de eventos	un único listener para los botones internos de la lista
`target` y `closest()`	identificación del elemento realmente interactuado
Actualización visual	rerenderizado tras cambiar estado o eliminar una tarea

---

16. Errores frecuentes al comenzar

> ⚠ **Errores muy habituales**
>
> Estos fallos aparecen con frecuencia al empezar a trabajar con interacción y eventos en el DOM.

16.1. Ejecutar una función al asignarla en lugar de pasarla como callback

```
const boton = document.querySelector("#boton");

// Incorrecto si querías que se ejecute al hacer clic
boton.addEventListener("click", mostrarMensaje());
```

En ese caso la función se ejecuta inmediatamente.

16.2. No diferenciar entre el elemento escuchado y el elemento que originó el evento

Cuando trabajas con listas o estructuras anidadas, no distinguir entre `currentTarget` y `target` genera mucho código confuso.

16.3. Añadir listeners individuales a muchos elementos cuando la delegación sería más apropiada

Esta estrategia puede funcionar al principio, pero se vuelve menos mantenible cuando la interfaz cambia dinámicamente.

16.4. Usar `stopPropagation()` sin comprender el flujo real del evento

A veces se usa para “hacer que funcione”, cuando en realidad el problema está en una mala organización de la interacción.

16.5. Mezclar cambios de datos y cambios visuales sin estructura mínima

Si cada listener modifica de forma dispersa múltiples partes del DOM y además altera datos internos sin criterio, el resultado se vuelve difícil de mantener.

16.6. No comprobar si el elemento objetivo es realmente el que buscas

En delegación de eventos conviene verificar siempre si el evento proviene de un elemento relevante antes de actuar.

---

17. Síntesis de la sesión

En esta sesión has trabajado el mecanismo que da vida a la interfaz:

- qué es un evento y cómo lo escucha JavaScript;
- cómo utilizar `addEventListener()`;
- qué papel juega el objeto evento;
- cómo interpretar `target` y `currentTarget`;
- cómo funciona la propagación;
- para qué sirve `stopPropagation()`;
- qué es la delegación de eventos y por qué es tan útil;

- cómo gestionar interacción sobre elementos dinámicos;
- cómo alternar estados visuales con `classList.toggle()`;
- cómo separar datos, eventos y renderizado;
- qué son los eventos personalizados;
- cómo empezar a dotar de comportamiento a una interfaz renderizada dinámicamente.

Esta sesión es decisiva porque te permite pasar de una interfaz generada dinámicamente a una interfaz que además reacciona con criterio. Ese paso es imprescindible antes de trabajar con formularios, validación, persistencia y, más adelante, con el enfoque declarativo de React.

✓ **Al terminar esta sesión deberías ser capaz de...**

- explicar qué es un evento en el navegador;
- escuchar eventos con `addEventListener()`;
- utilizar el objeto evento para interpretar una interacción;
- distinguir `target` de `currentTarget`;
- comprender la idea general de propagación y burbujeo;
- aplicar delegación de eventos en listas o estructuras dinámicas;
- alternar estados visuales con `classList.toggle()`;
- organizar una interacción sencilla separando evento, lógica y actualización visual.

---

18. Referencias documentales generales

Referencias oficiales y de consulta

- MDN Web Docs — Introduction to events
- MDN Web Docs — Event
- MDN Web Docs — `EventTarget.addEventListener()`
- MDN Web Docs — Event bubbling
- MDN Web Docs — Event capture
- MDN Web Docs — `Event.target`
- MDN Web Docs — `Event.currentTarget`
- MDN Web Docs — `Event.stopPropagation()`
- MDN Web Docs — Element: `click` event
- MDN Web Docs — Element: `input` event
- MDN Web Docs — HTMLElement: `change` event

- MDN Web Docs — HTMLFormElement: `submit` event
- MDN Web Docs — `Element.classList`
- MDN Web Docs — `Element.closest()`
- MDN Web Docs — `CustomEvent`
- MDN Web Docs — `EventTarget.dispatchEvent()`

Recomendación de estudio

Al repasar esta sesión, no te limites a comprobar que “el clic funciona”. Intenta describir siempre qué ha ocurrido exactamente: qué elemento escucha, qué elemento origina el evento, qué lógica decides aplicar y qué parte de la interfaz cambias como consecuencia. Esa forma de analizar la interacción te ayudará muchísimo en formularios, miniaplicaciones dinámicas y React.